Advanced Empirical Finance: Topics and Data Science

Welcome, setup and financial data

Stefan Voigt

University of Copenhagen and Danish Finance Institute (DFI)

Spring 2027

Welcome!

Who is in the room?

A short anonymous survey in the first lecture:

- Who are you: master's, exchange, or PhD student?
- How good are you at R, Python, and AI tools for coding?
- Which tools have you used?
- What do you mainly want to take away from this course?

What AEF is about

This course is about doing empirical finance: working with real data, from start to finish.

Theory	**Econometrics**	**Data & code**
What should hold	How to test it	What we actually find

- AEF lives where the three meet: most results depend on choices made between the raw data and the final table
- Empirical finance is finance measured against reality, and financial markets provide vast data to inform financial decisions

(How) can state-of-the-art methods improve financial decision making?

Why I teach this course

Decisions with large data
Portfolio choice for thousands of assets under transaction costs and estimation error (Hautsch & Voigt, 2019)

Reproducibility
How the choices researchers make change their results: non-standard errors (Gruber, Kastner, Voigt & Weiss, 2026; Voigt & Scheuch, 2026)

Tidy Finance
The open-source book with R and Python side by side that this course builds on: tidy-finance.org

One thread through the course

One object all semester: the **mean-variance efficient portfolio**, which is also the **stochastic discount factor** m that prices every asset, $1 = E[mR]$

From portfolio choice to pricing

Lectures 2–9

The efficient portfolio and its estimation error; shrinkage and transaction costs; the SDF and the CAPM; factors, sorts and reproducibility

Machine learning and the SDF

Lectures 10–16

Overfitting and out-of-sample evaluation; shrinking the SDF; latent factors; trees, forests and neural networks on the factor zoo

Volatility, risk and uncertainty

Lectures 17–21

Volatility and covariances from high-frequency data; covariance forecasts for portfolio choice; the conference session

Your roadmap

8 Feb	**Kick-off** – Trading game dry run until 12 Feb
16 Feb	**Trading starts**
11–18 Mar	**Assignment 1** – Portfolio choice
19–26 Apr	**Assignment 2** – Machine learning, out of sample
3–10 May	**Assignment 3**
14 May	**Trading game closes**
after 20 May	**Conference session**

Easter break 22–26 March. The lecture plan has every date.

How the course works

Every week

- **Lectures:** Mondays 8–10 in even weeks (CSS 1-1-18) and Thursdays 13–15 (CSS 18-01-11): methods, theory and recent papers
- **Exercises**, one page per week, in two parts:
  - *Reproduce* a chapter of Tidy Finance in R or Python; the solutions are in the book
  - *An open question* without a fixed solution: you submit a figure or a number, and the next lecture discusses the results of the whole class
- **Exercise classes:** open office hours, Mondays 8–10 in odd weeks (CSS 2-2-36). Bring your questions; nothing is presented

The learning curve is steep: ask early, on Absalon or in the exercise classes

How you are assessed

Continuous assessment: what you hand in and present during the semester counts

- **Three mandatory assignments**, in teams of up to three
- **Peer feedback** on the assignments of other teams, individually
- **The conference session:** presentations and discussions

TBD: weights and minimum requirements

Mandatory assignments

	Topic	Weeks
1	Portfolio choice	11–18 March
2	Machine learning in asset pricing, out of sample	19–26 April
3	**TBD**	3–10 May

- One week each; the assignments and the data come through Absalon
- Hand in one `.qmd` file and the PDF rendered from it: at most 7 pages, 12 pt, in English, from the templates
- Every assignment builds on your team's trading game decisions in one part
- The bar is high: the assignment quality criteria say what a good submission looks like

Peer feedback

- Peer feedback opens two days after the deadline and stays open for seven days
- Each of you gives feedback on **two** submissions of other teams
- Start by reproducing their report: does it render from their files, and do the numbers match?
- This is one of the most useful steps of the course: you see how others solved the same problem

TBD: confirm the rules for 2027

The trading game

The 60-second introduction on the homepage: aef-course.ku.dk/trading-game

The trading game: what matters

- Your team runs a paper portfolio of US stocks, from $1,000,000, from 16 February to 14 May; the dry run is open until 12 February
- You send a weight vector with a rationale whenever you like; the game trades it at the next open
- The game is there to make you decide and to discuss decisions: every assignment builds on your team's decisions
- **Conference session:** the teams with the best out-of-sample performance present their approach, and other students discuss it in front of faculty and quantitative traders; a jury picks the best team
- Start here: the rules and Getting started

AI in this course

- You may, and you are expected to, use AI assistants for code, text and review
- In exchange, the bar is high: you answer for every line you hand in
- The course repository contains `AGENTS.md`: your assistant reads the course rules from there
- Never paste licensed data (CRSP) or your trading game token into a chat
- How to work with an assistant: AI assistants

Team and contact

- Stefan Voigt (stefan.voigt@econ.ku.dk, www.voigtstefan.me)
- Teaching assistant: Nikolai Zakarias Zwisler (xjb522@econ.ku.dk)
- Questions about content go to the Absalon discussion board, so everyone benefits from the answer
- **TBD:** office hours, and whether lectures are recorded
- The rest of the team: your peers. Connect and help each other

Your setup

The toolbox

Tool	What it does
R or Python	The language you write your analysis in
Positron	The editor: code, console, data viewer, Git and AI assistant in one place
Quarto	Weaves text and code into one report: your hand-ins are Quarto files rendered to PDF
uv and uvr	Install Python, R and exactly the course's package versions
Git and GitHub	Version control and your team's repository

Set it up before the next lecture: technical prerequisites

One environment for everyone

- The course repository records every package version: `uv.lock` for Python, `uvr.lock` for R
- Two commands install exactly these versions on your computer:

```
uvr sync
uv sync
```

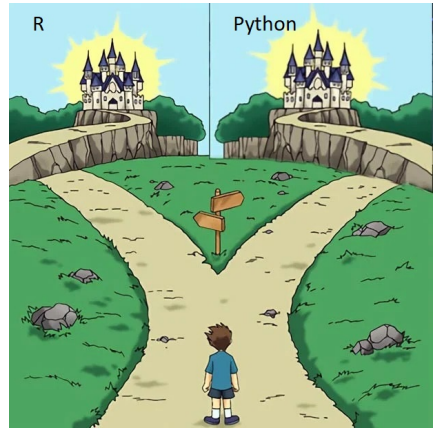
- Why it matters: your peers run your code for peer feedback; with the same versions, it runs on their computer too
- Need another package? `uvr add <package>` or `uv add <package>`, then commit the lock files

Your team repository

- Each team gets a private copy of the course repository on GitHub, created for you: course material, assignments and trading game code in one place
- Send your team's GitHub usernames through the form on Absalon
- Before every hand-in, a teammate renders your report from a fresh copy: the pull request is the natural place for this check
- Collaboration with Git and GitHub has every step

R or Python?

- Both are open source, have large communities and are standard in finance
- Pick one; every exercise and every solution exists in both
- **Tidy coding** matters more than the language: clean code in either is readable for everyone
- You give feedback on R and Python submissions: with tidy code, you understand your peers' code even in the other language



Financial data

Where the data come from

- **CRSP** (Center for Research in Security Prices): every US common stock since 1926, from WRDS. Licensed: you get it through Absalon
- **Public data**, downloaded with `tidyfinance`: the Fama-French factors and industry portfolios, the macroeconomic predictors of Welch and Goyal (2008), the consumer price index
- Everything lives as parquet files in the `data` folder of the course repository, as in the Tidy Finance book

File	Content
`crsp_monthly.parquet`	CRSP monthly returns, market capitalization, exchange, industry (from Absalon)
`factors_ff3_monthly.parquet`	Fama-French three factors and the risk-free rate
`industries_ff_monthly.parquet`	Ten industry portfolios
`macro_predictors.parquet`	Welch and Goyal (2008) predictors
`cpi_monthly.parquet`	Consumer price index

Reading the data (R)

```r
crsp_monthly <- read_parquet("../data/crsp_monthly.parquet")
cpi_monthly <- read_parquet("../data/cpi_monthly.parquet")
```

Reading the data (Python)

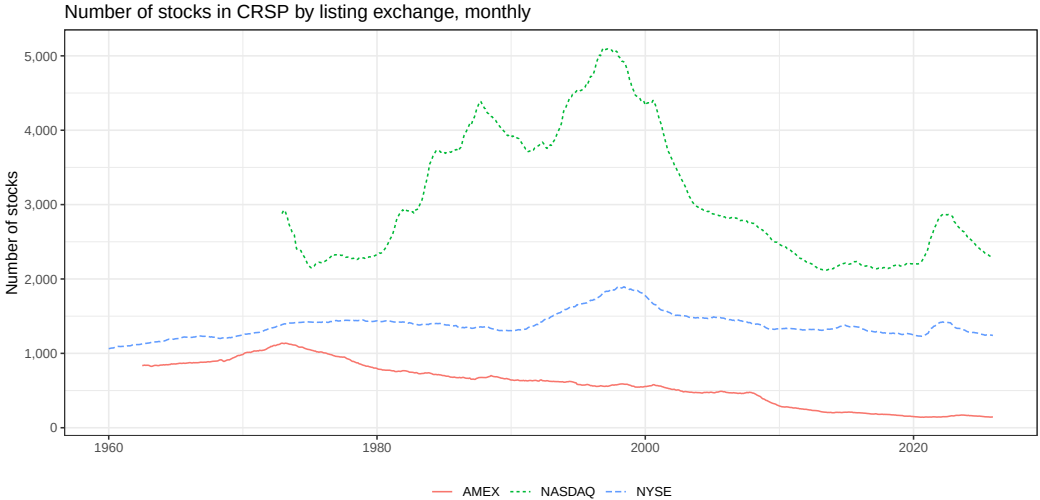
```
crsp_monthly = pl.read_parquet("../data/crsp_monthly.parquet")
cpi_monthly = pl.read_parquet("../data/cpi_monthly.parquet")
```

- The CRSP file is licensed: never commit it, never upload it, never paste it into an AI tool. Git ignores it in the course repository
- Read files with relative paths, so your code runs on your teammates' computers too

The CRSP universe: number of stocks (R)

```r
crsp_monthly |>
  count(exchange, date) |>
  ggplot(aes(x = date, y = n, color = exchange, linetype = exchange)) +
  geom_line() +
  labs(
    x = NULL, y = "Number of stocks", color = NULL, linetype = NULL,
    title = "Number of stocks in CRSP by listing exchange, monthly"
  ) +
  scale_y_continuous(labels = comma) +
  theme_bw() +
  theme(legend.position = "bottom")
```

The CRSP universe: number of stocks (R)



The CRSP universe: number of stocks (Python)

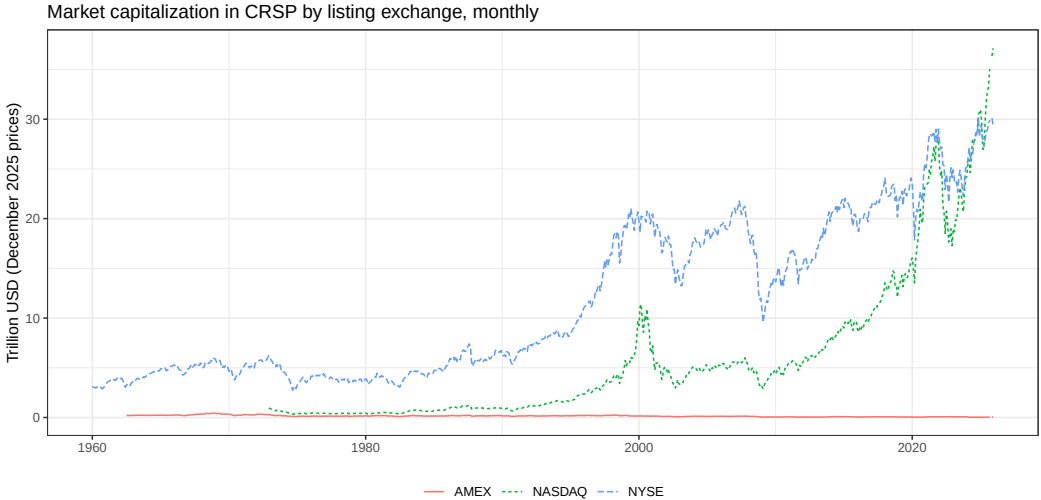
```
(
    ggplot(
        crsp_monthly.group_by("exchange", "date").len(),
        aes(x="date", y="len", color="exchange", linetype="exchange"),
    )
    + geom_line()
    + labs(
        x="", y="Number of stocks", color="", linetype="",
        title="Number of stocks in CRSP by listing exchange, monthly",
    )
    + scale_y_continuous(labels=comma_format())
    + theme_bw()
    + theme(legend_position="bottom")
)
```

Your first exercise: reproduce this figure from your own `data` folder

The CRSP universe: market capitalization (R)

```r
# mktcap is in million USD; cpi is 1 in December 2025
crsp_monthly |>
  left_join(cpi_monthly |> select(date, cpi), join_by(date)) |>
  summarize(mktcap = sum(mktcap, na.rm = TRUE) / first(cpi), .by = c(date, exchange)) |>
  ggplot(aes(x = date, y = mktcap / 1e6, color = exchange, linetype = exchange)) +
  geom_line() +
  labs(
    x = NULL, y = "Trillion USD (December 2025 prices)", color = NULL, linetype = NULL,
    title = "Market capitalization in CRSP by listing exchange, monthly"
  ) +
  theme_bw() +
  theme(legend.position = "bottom")
```

The CRSP universe: market capitalization (R)



The CRSP universe: market capitalization (Python)

```
# mktcap is in million USD; cpi is 1 in December 2025
market_cap = (
    crsp_monthly.join(cpi_monthly.select("date", "cpi"), on="date", how="left")
    .group_by("date", "exchange")
    .agg(mktcap=(pl.col("mktcap").sum() / pl.col("cpi").first()) / 1e6)
)
(
   ggplot(market_cap, aes(x="date", y="mktcap", color="exchange", linetype="exchange"))
    + geom_line()
    + labs(
        x="", y="Trillion USD (December 2025 prices)", color="", linetype="",
        title="Market capitalization in CRSP by listing exchange, monthly",
    )
    + theme_bw()
    + theme(legend_position="bottom")
)
```

Tidy coding: the essentials

Tidy data and the pipe

- **Tidy data:** each variable is a column, each observation a row, each value a cell. CRSP is tidy: one row per stock and month
- **One verb per step, chained:** `|>` in R, method chaining in Python. The code reads like the analysis

```r
# R: stocks per exchange in the latest month
crsp_monthly |>
  filter(date == max(date)) |>
  count(exchange)
```

```
# A data frame: 3 x 2
  exchange     n
  <chr>    <int>
1 AMEX       146
2 NASDAQ    2298
3 NYSE      1241
```

```python
# Python: stocks per exchange in the latest month
(
   crsp_monthly.filter(pl.col("date") == pl.col("date").max())
    .group_by("exchange")
    .len()
    .sort("exchange")
)
```

```
shape: (3, 2)

  exchange   len
  ---        ---
  str        u32

  AMEX       146
  NASDAQ     2298
  NYSE       1241
```

- Names say what a column holds (`trading_volume_usd`, not `tmp2`); the quality criteria have the rest

A first case study: daily returns (R)

```r
prices <- download_data(
  domain = "Stock Prices",
  symbols = symbols,
  start_date = start_date,
  end_date = end_date
)

returns <- prices |>
  arrange(symbol, date) |>
  mutate(ret = adjusted_close / lag(adjusted_close) - 1, .by = symbol) |>
  select(symbol, date, ret) |>
  drop_na(ret)
```

A first case study: daily returns (Python)

```
prices = tf.download_data(
    domain="Stock Prices",
    symbols=symbols,
    start_date=start_date,
    end_date=end_date,
)

returns = (
    prices.sort("symbol", "date")
    .with_columns(ret=pl.col("adjusted_close").pct_change().over("symbol"))
    .select("symbol", "date", "ret")
    .drop_nulls("ret")
)
```

Summaries by group (R)

```r
returns |>
  summarize(
    mean = mean(ret),
    sd = sd(ret),
    min = min(ret),
    max = max(ret),
    .by = symbol
  )
```

```
# A tibble: 2 x 5
  symbol     mean     sd    min   max
  <chr>     <dbl>  <dbl>  <dbl> <dbl>
1 AAPL   0.00119  0.0243 -0.519 0.153
2 MSFT   0.000578 0.0189 -0.156 0.196
```

Summaries by group (Python)

```
returns.group_by("symbol").agg(
    mean=pl.col("ret").mean(),
    sd=pl.col("ret").std(),
    min=pl.col("ret").min(),
    max=pl.col("ret").max(),
).sort("symbol")
```

```
shape: (2, 5)

  symbol   mean       sd         min         max
  ---      ---        ---        ---         ---
  str      f64        f64        f64         f64

  AAPL     0.001191   0.024263   -0.518692   0.153289
  MSFT     0.000578   0.018901   -0.155978   0.195652
```

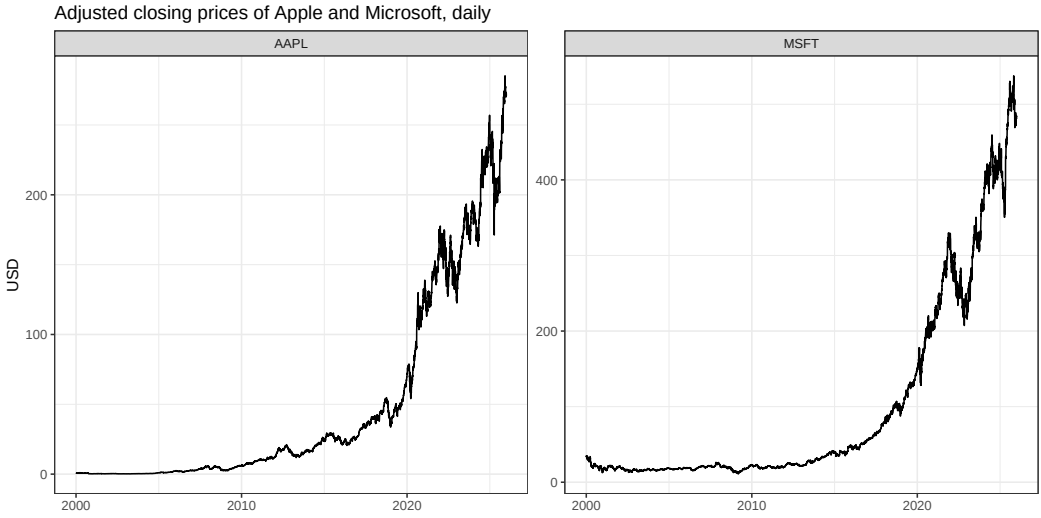
`group_by` and `summarize` (or `agg`) answer many questions in empirical finance

The grammar of graphics (R)

Data, a geometric shape, and aesthetics that map variables to it: `ggplot2` in R, `plotnine` in Python

```
prices |>
  ggplot(aes(x = date, y = adjusted_close)) +
  geom_line() +
  facet_wrap(~symbol, scales = "free_y") +
 labs(x = NULL, y = "USD", title = "Adjusted closing prices of Apple and Microsoft, daily") +
  theme_bw()
```

The grammar of graphics (R)



The grammar of graphics (Python)

```
(
    ggplot(prices, aes(x="date", y="adjusted_close"))
    + geom_line()
    + facet_wrap("symbol", scales="free_y")
    + labs(x="", y="USD", title="Adjusted closing prices of Apple and Microsoft, daily")
    + theme_bw()
)
```

Wrapping up

Who is in the room: your answers

The answers appear here after the first lecture.

Before the next lecture

1. **Setup:** Positron, uv and uvr, `uvr sync` and `uv sync`, TinyTeX: the technical prerequisites
2. **Data:** put `crsp_monthly.parquet` from Absalon into the `data` folder of the course repository
3. **Exercises:** Introduction to Tidy Finance: reproduce the number of stocks by exchange, then the open question: one figure of your own
4. **Team:** form your team of up to three; send your GitHub usernames through the form on Absalon
5. **Trading game:** try the tools in the dry run (open until 12 February). **TBD:** how teams get their token

Where to get help

- **Absalon discussion board:** questions about content and code, so everyone benefits from the answer
- **Exercise classes:** open office hours, Mondays 8–10 in odd weeks
- **Your AI assistant:** with the course rules from `AGENTS.md`; you check what it writes
- **The homepage:** aef-course.ku.dk has the lecture plan, the exercises, the trading game and the guides